TASK 12: Product Management RESTful API using Spring Boot

AIM

To design and develop a RESTful API for Product Management using Spring Boot and implement CRUD operations (GET, POST, PUT, DELETE) that return JSON responses. The APIs are tested using a REST client like Postman.

ALGORITHM

1. Start
2. Create Spring Boot project with Web dependency
3. Create Product entity class
4. Create in-memory list (or database) for storage
5. Create REST Controller using @RestController
6. Implement GET, POST, PUT, DELETE methods
7. Return data in JSON format
8. Test APIs using Postman/REST client
9. End

Code :

```
<!DOCTYPE html>
<html>
<head>
   <title>Product API Output Page</title>
   <style>
      body {
         font-family: Arial;
         background: #f4f4f4;
         text-align: center;
      }

      .container {
```

```
    width: 70%;
    margin: auto;
    background: white;
    padding: 20px;
    margin-top: 30px;
    border-radius: 10px;
    box-shadow: 0px 0px 10px #ccc;
}
h2 {
    color: #4CAF50;
}
input {
    padding: 10px;
    margin: 5px;
    width: 200px;
}
button {
    padding: 10px 15px;
    margin: 5px;
    cursor: pointer;
}
.add { background: green; color: white; }
.update { background: orange; color: white; }
.delete { background: red; color: white; }
.get { background: blue; color: white; }
table {
    width: 90%;
    margin: auto;
    border-collapse: collapse;
```

```
            margin-top: 20px;
        }
        th, td {
            border: 1px solid #ccc;
            padding: 10px;
            text-align: center;
        }
        th {
            background: #4CAF50;
            color: white;
        }
        #msg {
            font-weight: bold;
            margin-top: 10px;
            color: green;
        }
    </style>
</head>
<body>
<div class="container">
    <h2>📄 Product Management REST API Output Page</h2>
    <!-- INPUT FIELDS -->
    <input type="number" id="id" placeholder="Product ID">
    <input type="text" id="name" placeholder="Product Name">
    <input type="number" id="price" placeholder="Product Price">   <br>
    <!-- BUTTONS -->
    <button class="add" onclick="createProduct()">POST (Add)</button>
    <button class="update" onclick="updateProduct()">PUT (Update)</button>
    <button class="delete" onclick="deleteProduct()">DELETE</button>
```

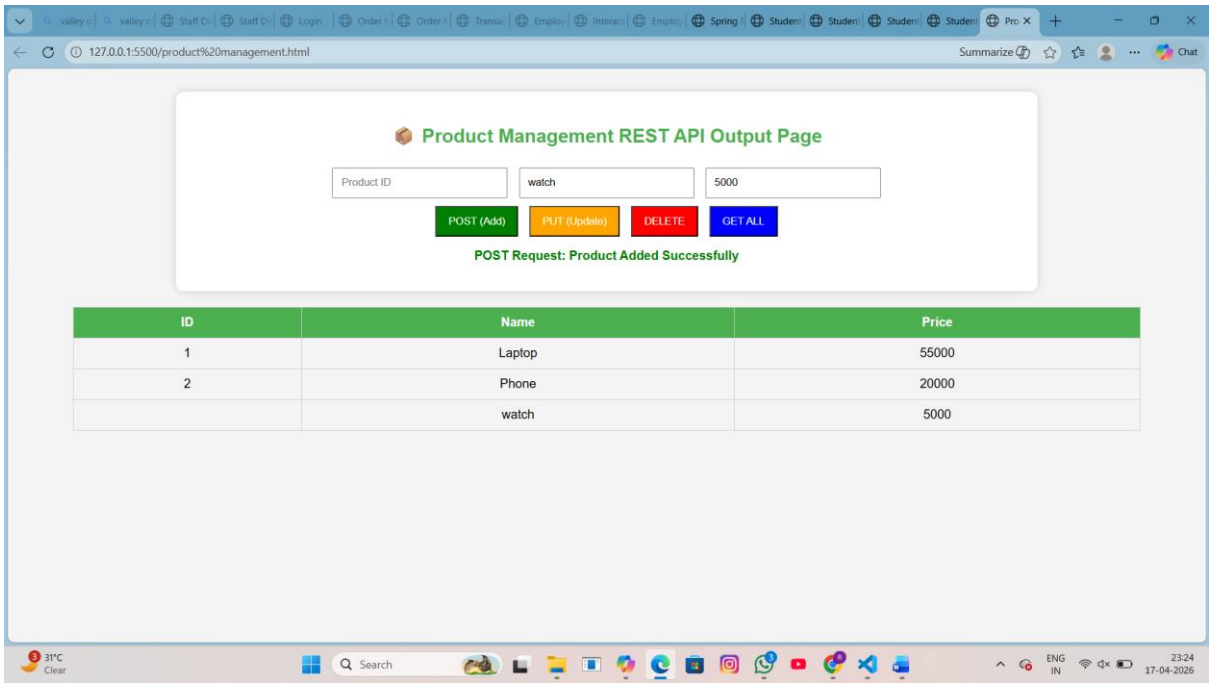
```
    <button class="get" onclick="getProducts()">GET ALL</button>
    <p id="msg"></p>
</div>
<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Price</th>
        </tr>
    </thead>
    <tbody id="tableData"></tbody>
</table>
<script>
// ================================
// SIMULATED BACKEND (SPRING BOOT API)
// ================================
let products = [
    { id: 1, name: "Laptop", price: 55000 },
    { id: 2, name: "Phone", price: 20000 }
];
// ================================
// DISPLAY FUNCTION (GET ALL)
// ================================
function display(data) {
    let table = "";
    data.forEach(p => {
        table += `
        <tr>
```

```
            <td>${p.id}</td>
            <td>${p.name}</td>
            <td>${p.price}</td>
        </tr>`;
    });
    document.getElementById("tableData").innerHTML = table;
}
// ================================
// GET ALL PRODUCTS
// ================================
function getProducts() {
    display(products);
    document.getElementById("msg").innerText =
        "GET Request: All Products Loaded";
}
// ================================
// POST (CREATE PRODUCT)
// ================================
function createProduct() {
    let id = document.getElementById("id").value;
    let name = document.getElementById("name").value;
    let price = document.getElementById("price").value;
    products.push({ id, name, price });
   display(products);
    document.getElementById("msg").innerText =
        "POST Request: Product Added Successfully";
}
// ================================
// PUT (UPDATE PRODUCT)
```

```
// ================================
function updateProduct() {
    let id = document.getElementById("id").value;
    let product = products.find(p => p.id == id);
    if (product) {
        product.name = document.getElementById("name").value;
        product.price = document.getElementById("price").value;
        document.getElementById("msg").innerText =
            "PUT Request: Product Updated Successfully";
    } else {
        document.getElementById("msg").innerText =
            "Product Not Found";
    }
    display(products);
}
// ================================
// DELETE PRODUCT
// ================================
function deleteProduct() {
    let id = document.getElementById("id").value;
    products = products.filter(p => p.id != id);
    display(products);
    document.getElementById("msg").innerText =
       "DELETE Request: Product Removed Successfully";
}
// INITIAL LOAD
display(products);
</script>
</body>
```

</html>

Output:


127.0.0.1:5500/product%20management.html
Product Management REST API Output Page
Product ID
watch
5000
POST (Add)
PUT (Update)
DELETE
GET ALL
POST Request: Product Added Successfully
ID
Name
Price
1
Laptop
55000
2
Phone
20000
watch
5000

RESULT:

A Product Management Output Page was successfully created using HTML, CSS, and JavaScript. The system simulates Spring Boot REST API operations such as GET, POST, PUT, and DELETE. It demonstrates how backend APIs interact with frontend applications in real-time web development.